

**Name:** Sam Setta  
**Date:** May 26, 2026  
**Course:** Foundations Of Python Programming  
**Assignment:** Assignment 06  
**Github URL:**

## Creating functions and organizing code with SoC

### Introduction

In this document, I outline how to build upon the program to enter student course registration data by adding functions for each step and organizing the code by separation of concerns (SoC). Each function from the main body of the starter script (Assignment 05 program) was moved to one of two classes, and the code referencing each task was replaced with the function call. The functions were split into two classes: either file processing (reading and writing file data) or Input/Output (I/O). Program testing included running the script after each function was moved, and testing the final script to ensure functionality was the same as the Assignment 05 program.

### Creating a Python script for the program

#### *Opening python script in PyCharm and editing the header*

Similar to module 05, a new project was created in the module 06 directory before opening the starter script (Assignment06-starter.py), saving to Assignment06.py and editing the header with my name and date.

#### *Load modules and define the Data Constants and Variables*

Many of the data variables used in Assignment 05 were removed for this program since most could be used as local variables within each function. While writing the script, I kept all the variables and data constants from the starter script, and removed them after functions were written. The data constants remained the same, and included the FILE\_NAME set to the 'Enrollment.json' file and MENU for the list of menu options. While the only variables necessary to run the script after SoC was the students variable set as an empty list, and the menu\_choice, set to an empty string.

#### *Main body of the script*

The structure of the Python program is very different from previous modules. Here, separation of concerns by file processing (FileProcessing) and input/output (IO) classes separates functions at the start of the script into these two classes. After which, the JSON module is loaded, and variables and data constants are loaded. Finally, the script

loads data from the JSON module before printing menu options, taking user input and executing based on user input, all done by calling functions defined at the beginning of the script.

## Separation of Concerns (SoC)

Each function within the file processing and input/output class included a docstring with a description of what it does, a change log, and a description of the return value (None if no value is returned). Here, I outline the order in which I moved tasks to functions within each class, before calling the functions later in the script. After each function was defined and the code accomplishing that task later in the script was replaced by the function call, I tested the program worked as expected using break points and the PyCharm debugger.

### *Input/Output (IO) Class*

#### Error message handling function

The first function moved to the classes now defined at the beginning of the script was the error message handling task, defined as the function `output_error_messages( )` using the `@staticmethod` decorator to indicate the static class of the function. Error message reporting was done throughout the script and would be replaced in many of the other parts of the script, so starting with this function allowed these pieces of code to instead be replaced with a single function. The parameters included in this function were the message string, which could be provided with a custom string as an argument specific to the error message for that part of the code, and an error exception set to a default value of None. The function first prints the error message, before checking that the default error value has been replaced by an argument and printing the technical message from the error handling if so (see code below).

```
print(message, end="\n\n")
if error is not None:
    print("-- Technical Error Message -- ")
    print(error, error.__doc__, type(error), sep='\n')
```

Once the `output_error_message` function was defined, all of the error handling later in the script was replaced with this function by using the drill through function with the class (`IO.output_error_message`). Depending on the error for each step, the custom error message argument is set to the error message string parameter and error called using the `e` variable from the try-except code (see example below).

```
IO.output_error_messages("Text file must exist before running this
script!\n",e)
```

This function replaced the two types of errors when reading the data from a file, saving the data to a file, entering menu options that are not one of the four options, and entering numbers into the student name or course name. By writing this function first, the error handling throughout the program could be replaced with `output_error_message()` function and tested all at once.

### Output menu function

The code related to printing the menu to the display was moved to the `output_menu()` function, with an input parameter of the menu message string to be printed. Moving the code to a function here, allows adaptability later in the evolution of the program if the printed menu needs to be edited, only the MENU constant needs to change. Similar to lab 3, I used two print statements, one before and one after the `print(menu)` function to put space before and after the printed menu. Here, the only change was calling the menu parameter within the print function, rather than the MENU constant.

### Input menu choice function

The `input_menu_choice()` function was created to take user input for running code and store the input for the later if-else statement used in the main body of the code. For this function the original code related to assigning `menu_choice` was edited to include a try-except error handling statement in case something other than the available entries are entered. There are no parameters included for the function, since only the printed statement asking the user to 'choose an option' and the stored user input is needed. The choice parameter is set to user input with the `input()` function, with the printed message 'Choose a menu option: ' (see code below). The try-except code first tries taking user input and if the user input is not one of the four available strings (1,2,3,4), it raises an Exception which is handled in the exception block with the `output_error_messages()` function within the IO class (see code block). After the try-except block the function returns the choice (string of the user input). The default value for choice is 0, and will be reset with the user input.

```
choice = "0"
try:
    choice = input("Choose a menu option: ")
    if choice not in ("1", "2", "3", "4"): # Note these are strings
        raise Exception("Please, choose only 1, 2, 3, or 4")
except Exception as e:
    IO.output_error_messages(e.__str__(),e)
return choice
```

### Output student courses function

To print out registered student names and courses, the `output_student_courses()` function was created. This function keeps the same code from the starter script to print fifty dashes before and after cycling through each 'row' or entry within the `student_data` parameter to pull the 'FirstName', 'LastName', and 'CourseName' using the respective key names before printing the message that a student is registered. This code is useful for both printing the registered students info if that menu choice is selected, and printing the data that was saved to the `Enrollment.json` file when writing to the file (menu option 3).

### Input student data function

Finally, the last IO function included in this program was the `input_student_data` function, for which code was copied for taking user input of student first name, last name and course name, including error handling. Once again, the `output_error_message()` function simplifies the code and is used to print errors if any numbers are included in the student first or last name. Here the only parameter is the `student_data` string which is returned at the end of the function. Within the function the user input for student and course name is stored in a local variable of `student_entry` before appending to the `student_data` parameter.

### *File Processing Class*

There were two tasks from the program that deal with file processing: 1) reading file data from the 'Enrollment.json' file, and 2) saving data to the 'Enrollment.json' file.

### Reading data file from Enrollment.json

The code related to reading a file into the program at the beginning of the starter script was moved to the function `read_data_from_file( )` defined using the `@staticmethod` decorator in Python to indicate the static class of the function. The parameters for this function included the `file_name` (string) and `student_data` (list) that were later used to assign arguments when calling functions. The local variable, `file`, was set to 'None' before the try, except, finally block of code reading the data file into the program was copied to this function. The error code handling was already included by calling the `output_error_message()` from the IO class. The only other change that was made to the original code from the starter script was replacing the global `FILE_NAME` constants with the `file_name` parameter and saving the output of `json.load` to the `student_data` parameter (code below). The end of the function returns the `student_data` to the user (code below).

```
try:
    file = open(file_name, "r")
    student_data = json.load(file)
```

```

except Exception as e:
    IO.output_error_messages("Please check that the file exists and that it
    is in a json format.", e)
finally:
    # Check if a file object exists and is still open
    if file is not None and file.closed == False:
        file.close()
return student_data

```

### Writing data to the Enrollment.json file

The write\_data\_to\_file() function used to save data to the Enrollment.json file was defined similarly to the read\_data\_from\_file() function. The file local variable was set to none, before the try, except, finally code block was moved to the function, and the FILE\_NAME replaced with the file\_name parameter before returning the student\_data. The error handling was done by the output\_error\_message() function from the IO class.

## Main program code (putting it all together)

Once the functions are defined within each class the main program code is much cleaner and simpler. In this part of the script the students variable is set to the FileProvessor function read\_data\_from\_file, with the FILENAME constant argument set to the file\_name parameter, and the students variable argument set to the student\_data parameter. This reads in the student data from the Enrollment.json file and saves it to the students variable so we can use it later to register more students or print the registered students. The while (True) statement is then used to start iterating through menu options, both the output\_menu() and input\_menu\_choice() functions are user to print menu options and take user input. The MENU constant is used as the argument for the menu parameter for output\_menu, while the input\_menu\_choice() function is set to menu\_choice to save the user input and iterate through program options.

The respective functions are used within the if-else statements to move through the options. Option 1 calls the input\_student\_data() function within the IO class setting the student\_data parameter to the students variable. Option 2 calls the output\_student\_course function using the students variable for input to the student\_data parameter. Option 3 uses both the write\_data\_to\_file function with the FILENAME constant supplied to the file\_name parameter, and students to the student\_data parameter, followed by a printed message that the 'following data was saved to a file:' before calling the output\_student\_courses function using the students argument for the student\_data parameter. Option 4 remains the same.

## Testing the program

### *Testing program in PyCharm and the command line*

I tested the program in both PyCharm and command line. While working in PyCharm, I was already in the module 06 project, but after opening command line I first navigated to my python course directory and module 05 directory within Terminal in MacOS. I ran the program code in python with the following code: `python3 Assignment06.py`. I tested the program in both PyCharm and command line with the same steps that are outlined below but included the different output of each from each program in screenshots.

To test the program takes user input, I started the program and input option 1, where I entered student registration information (first, last and course name). Afterwards I selected option 2, which displayed the data from the data file and the new entry (Vic, Vu, Python201), before selecting option 3, and getting the expected, 'Data Saved!', message (Figure 1) along with the student data that was saved including Vic Vu Python201. Finally, I selected option 4 to exit the program, which exited as expected.

```
---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----

Choose a menu option: 1
Enter the student's first name: Vic
Enter the student's last name: Vu
Please enter the name of the course: Python201
You have registered Vic Vu for Python201.

---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----

Choose a menu option: 2

Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Su Jones is enrolled in Statistics
Student Vic Vu is enrolled in Python201

---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----

Choose a menu option: 3
The following data was saved to file:

Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Su Jones is enrolled in Statistics
Student Vic Vu is enrolled in Python201

---- Course Registration Program ----
Select from the following menu:
  1. Register a Student for a Course.
  2. Show current data.
  3. Save data to a file.
  4. Exit the program.
-----
```

**Figure 1.** Screenshot of user entry when selecting option 1, present the current data including the newly entered information, and saving the data to a file in PyCharm (left) and command-line (right).

To check the user input was saved properly, the Enrollment.json file was opened in both PyCharm and visual studio code (VS code) (Figure 3). Both had the expected student data and newly registered student (Figure 2).

```
[
  {
    "FirstName": "Bob",
    "LastName": "Smith",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Sue",
    "LastName": "Jones",
    "CourseName": "Python 100"
  },
  {
    "FirstName": "Su",
    "LastName": "Jones",
    "CourseName": "Statistics"
  },
  {
    "FirstName": "Vic",
    "LastName": "Vu",
    "CourseName": "Python201"
  }
]
```

```
Assignment > {} Enrollments.json > ...
1
2
3 {
4   "FirstName": "Bob",
5   "LastName": "Smith",
6   "CourseName": "Python 100"
7 },
8 {
9   "FirstName": "Sue",
10  "LastName": "Jones",
11  "CourseName": "Python 100"
12 },
13 {
14   "FirstName": "Su",
15   "LastName": "Jones",
16   "CourseName": "Statistics"
17 },
18 {
19   "FirstName": "Vic",
20   "LastName": "Vu",
21   "CourseName": "Python201"
22 }
```

**Figure 2.** Screenshot of Enrollment.json file after saving new entry in PyCharm (left) and VS code (right).

To test if the program allows multiple registrations from user input and that these entries were saved to the JSON file, I restarted the program in both PyCharm and command-line. I selected option 1, and added a new student name and course name, before repeating the process to ensure I could add multiple entries (Figure 4). I checked the new data would be presented to the user, I selected option 2 and data was printed as expected, before choosing option 3 which notified me that the data was saved (Figure 4). To double-check the file data was saved properly, I re-opened the Enrollment.json file in both PyCharm and VS code and saw the newly entered data in the file.



```

Choose a menu option: 1
Enter the student's first name: Vic
Enter the student's last name: Vu
Please enter the name of the course: Python201
You have registered Vic Vu for Python201.

---- Course Registration Program ----
Select from the following menu:
1. Register a Student for a Course.
2. Show current data.
3. Save data to a file.
4. Exit the program.
-----

Choose a menu option: 1
Enter the student's first name: Henry
Enter the student's last name: Hank
Please enter the name of the course: Python 101
You have registered Henry Hank for Python 101.

---- Course Registration Program ----
Select from the following menu:
1. Register a Student for a Course.
2. Show current data.
3. Save data to a file.
4. Exit the program.
-----

Choose a menu option: 2
-----
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Su Jones is enrolled in Statistics
Student Vic Vu is enrolled in Python201
Student Henry Hank is enrolled in Python 101
-----

---- Course Registration Program ----
Select from the following menu:
1. Register a Student for a Course.
2. Show current data.
3. Save data to a file.
4. Exit the program.
-----

Choose a menu option: 1
Enter the student's first name: Vic
Enter the student's last name: Vu
Please enter the name of the course: Python 201
You have registered Vic Vu for Python 201.

---- Course Registration Program ----
Select from the following menu:
1. Register a Student for a Course.
2. Show current data.
3. Save data to a file.
4. Exit the program.
-----

Choose a menu option: 2
-----
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Su Jones is enrolled in Statistics
Student Vic Vu is enrolled in Python 201
Student Henry Hank is enrolled in Python 100
-----

```

**Figure 3.** Screenshot of program testing to check if multiple student data entries code be entered and saved in PyCharm (left) and command-line (right).

## Summary

In this document, I've outlined how to use SoC, functions and classes to make the program from Assignment05.py more flexible and readable. Two classes are used to organize concerns, the FileProcessor class and IO (input/output) class within which tasks from the program are defined as functions. Two functions are defined within the FileProcessor class to read and write student data to the Enrollment.json file. While functions for error handling messages, menu printing, user menu input, user input for student registration, and displaying registered students are defined within the IO class. During development, each function was tested as the task was moved from the main body code to functions to ensure bugs were easily addressed. Finally, the program was

tested similarly to the program from Assignment05 ensuring multiple user entries would be added to student\_data and were loaded and saved based on user input.